turf.build

REFERENCE ARCHITECTURE · V0.1 · OCTOBER 2026

Pull Request to Production

A reference architecture for Terraform delivery on GitHub Agentic Workflows, with an agent in the pipeline.



For platform engineers who run Terraform through GitOps CI/CD.

THE PIPELINE

The pipeline you already have

Terraform® delivery through GitHub® Actions has a shape every platform team knows, and HashiCorp's own tutorial lays it out: a plan phase on every pull request, an apply phase on every push to `main`. Two things go wrong with it.

The reviewed plan is not the applied plan. In the tutorial's words, the plan workflow “marks it as speculative so it cannot modify infrastructure” and “adds a comment to the pull request with a link to the run”. A reviewer reads that plan and approves the merge. The apply workflow then “uploads the Terraform configuration” again, “creates a Terraform apply run using the configuration uploaded in the previous step”, and “confirms and applies the run”. That is a second plan, made minutes or days after the first, against whatever the account and `main` hold by then. Nothing in the pipeline ties the two plans together.

The apply is unattended. With a merge as its trigger, the apply phase runs with `-auto-approve`, which, as OpenTofu's documentation puts it, “Skips interactive approval of plan before applying.” Nobody reads the plan that actually runs. The providers it loads, plugins that hold the cloud credentials, are never checked. An unexpected replacement leads to an outage, and the first anyone hears of it is the alert.

> **Turf™ is a self-hosted, autonomous infrastructure-as-code tool, built on OpenTofu™.**

This paper puts an agent in the pipeline, with Turf as its engine. GitHub Agentic Workflows run the agent as a GitHub Actions job. Turf runs on the same runner and gives the agent a plan to read, an approval to give by digest, and a verify step to carry out from inside the apply. The reviewed plan is the plan that applies. The rules are paragraphs in the repository. And the apply is attended, by the agent, every time.

THE PLAN-AND-APPLY PIPELINE, AS HASHICORP'S TUTORIAL DRAWS IT



WITH AN AGENT IN THE PIPELINE

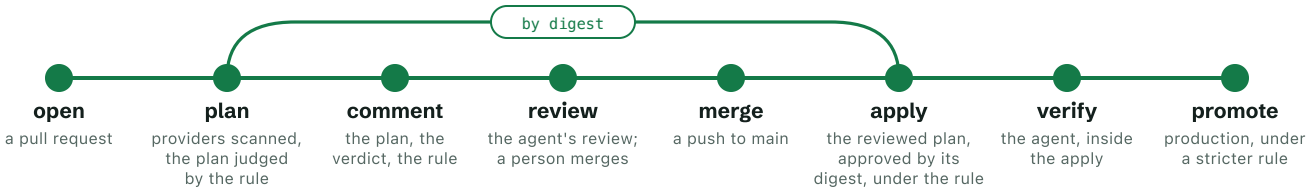
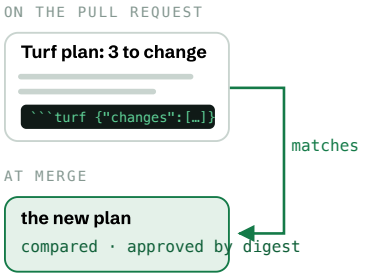


Figure 1. The same pipeline, twice. Above, as HashiCorp's tutorial draws it: two plans, and nothing between them. Below, with an agent in it.

At a glance

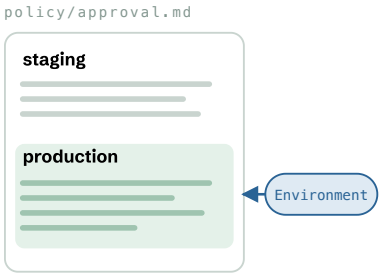
An agent in the pipeline, three ways.



The reviewed plan is the applied plan

The plan run posts its plan on the pull request, with the plan as data at its foot. The apply run plans again, compares, and approves by the new plan's digest only when it matches what was reviewed. A plan that differs is refused, and the report says what changed.

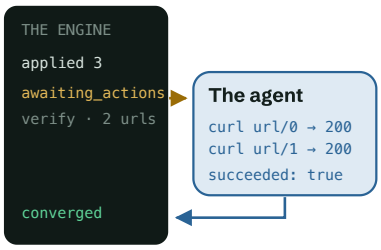
→ Chapters 3 and 5



Rules in plain language

The approval rule is a paragraph, one per GitHub Environment; the providers rule is another file. The agent reads the rule the Environment names and writes its verdict with the rule that decided, so the record reads the same to a person and to the next run.

→ Chapters 4 and 8



The agent inside the apply

A turf action is a step the configuration asks of the agent. Here the apply pauses after the release and asks the agent to fetch every function URL and check the answer. The agent does it with curl and reports back; the engine carries on, or halts.

→ Chapter 6

The loop in one line: a pull request → plan, scan, verdict → merge → staging: approve by the rule, apply, verify, report → production: the same, under a stricter rule. **Part I** is the host: GitHub Agentic Workflows, and Turf on the runner. **Part II** walks the loop, one stop at a time. **Part III** is what it guarantees.

PART I
THE HOST

01

OPEN
PLAN
COMMENT
REVIEW
MERGE
APPLY
VERIFY
PROMOTE

IN GITHUB'S WORDS

GitHub Agentic Workflows

GitHub's definition: “AI-powered repository automations that you define in markdown and run as GitHub Actions.” Generally available since October 28, 2026.

A workflow is one Markdown file: YAML frontmatter for the trigger, the permissions, the engine and the tools, then the prompt. `gh aw compile` turns it into an ordinary Actions workflow, a `.lock.yml`, and both are committed. The engine is GitHub Copilot by default; Claude and Codex are the alternatives. This paper's workflows run Claude Sonnet 5.5.

A run has a shape. The agent runs in a container behind the Agent Workflow Firewall, a domain allowlist, with a read-only token. Its tools come through an MCP gateway that enforces the tool list the frontmatter declares. It writes nothing to GitHub itself: its comment, review or label goes to safe outputs, which post it after a threat-detection pass. An HTTP MCP server on the runner is allowed, which is how Turf joins.

Three workflows make the pipeline: `turf-plan` on every pull request that touches `infra/`, `turf-apply-staging` on every push to `main`, and `turf-apply-production` when the staging run completes. They share one import, which starts Turf, and one runbook, read at run time.

```
---
on: pull_request affecting infra/**
environment: staging
engine:
  id: claude
  model: claude-sonnet-5-5
imports:
  - shared/turf.md
mcp-servers:
  turf:
    type: http
    url: http://localhost:3000/mcp
    allowed: [skill_deployment, read_skill_file, init, open, up, show, status, cancel]
safe-outputs:
  add-comment:
    max: 1
    hide-older-comments: true
  submit-pull-request-review:
    allowed-events: [COMMENT, REQUEST_CHANGES]
---

# Turf plan

You are the plan reviewer for a pull request. This run **plans only** …

{{#runtime-import .github/workflows/shared/turf-runbook.md}}
```

`.github/workflows/turf-plan.md`, trimmed. The runbook it imports is the same for all three workflows.

ON THE RUNNER

Turf on the runner

Turf runs where the agent runs: on the GitHub Actions runner, started before the agent and stopped after it. The agent sees an MCP server. The credentials never leave the host.

Three pre-agent steps do it. The first writes the run's context to disk: the Environment's name, the rule it names, the state bucket and key, the head commit, and the earlier plan comments on the pull request. The second installs Turf and restate-server. The third mints a one-hour AWS session for the Environment's role from the job's OIDC token and starts the Turf engine and the MCP server in that shell; the credentials are exported there and nowhere else. The agent container starts afterwards and reaches Turf through the gateway.

What the agent may call is the frontmatter's `allowed` list, enforced at the gateway: a plan run has `init`, `open`, `up`, `show`, `status` and `cancel`; an apply run adds `apply`, `output` and `state_list`. No run has `destroy`, `taint`, `import` or `invoke`. State lives in S3, one key per Environment, with its lock file; a post-step stops the server and the engine cleanly, so the lock is released.

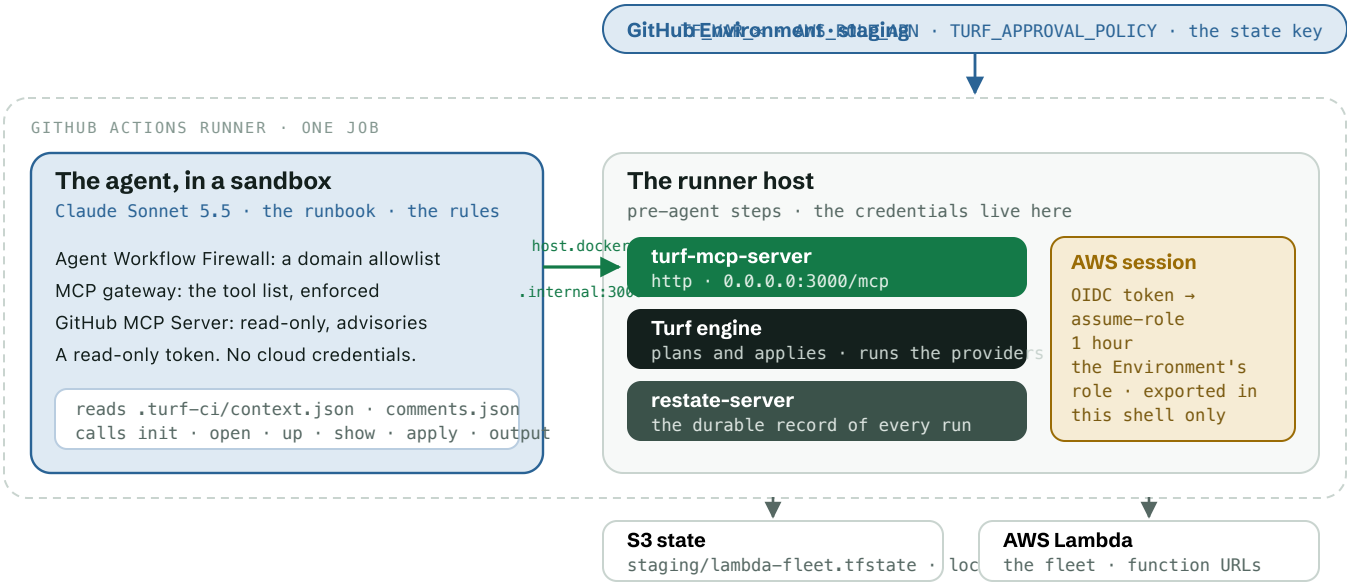


Figure 2. The runner. One job, two sides. The agent's side has the sandbox, the firewall and the gateway; the host's side has Turf and the cloud session. The Environment feeds the host, and what the agent learns of it arrives as files.

RUN, 2026-10-31	AGENT TIME	COST	TURNS	JOB
Plan, on the pull request	44 s	$0.178	13	2 min 21 s
Apply, staging	68 s	$0.254	18	2 min 43 s
Apply, production	84 s	$0.233	19	3 min 5 s

Measured on Claude Sonnet 5.5, from the runs' own summaries; the job column is the whole Actions job, Turf's start-up included. Run links in the Sources.

PART II
THE LOOP

03

- OPEN
- PLAN
- COMMENT
- REVIEW
- MERGE
- APPLY
- VERIFY
- PROMOTE

DAY 1 · THE PULL REQUEST

Plan, scan, verdict

A pull request that touches `infra/` gets a plan, a providers check and a verdict: one comment and one review, a few minutes after it opens.

The run plans against staging, the Environment the plan workflow binds. The agent calls `init`, which reports the providers the lock file resolves, and checks each against the providers rule before anything is opened: the source namespace, the pin and its hashes, and GitHub's Advisory Database, through the GitHub MCP Server, at the resolved version. Then `open` and `up`, which stop at an approval point with the plan. The agent lists every change with the attributes that differ and judges it by the staging rule.

The comment is the record: the providers table, the plan in a diff block, the verdict with the rule that decided, and a closing `turf` block with the plan as data. The review is `COMMENT` when the plan is ready to merge, `REQUEST_CHANGES` when a rule refused. Then the run cancels the convergence it made: a plan run holds nothing.

```
3. Advisories. For every provider, look up the GitHub Advisory Database for its module
   github.com/hashicorp/terraform-provider-<name> at the resolved version
   (list_global_security_advisories, ecosystem go, affects: "<module>@v<version>").
   - An advisory of severity high or critical whose fixed version is above the
     resolved version, or that has no fixed version, is a deny.
   - Anything of lower severity, or already fixed at or below the resolved version, is
     reported in the Providers table and does not block.
   - A lookup that fails is reported as "not checked"; it does not block.
```

policy/providers.md, rule 3. Rules 1 and 2 are the source namespace and the pin.

TOOL CALLS · TURF, THEN GITHUB

```
✓ init · infra
providers = [
  { hashicorp/archive  2.8.1  ✓ }
  { hashicorp/aws      6.71.0 ✓ }
]
✓ list_global_security_advisories
  ecosystem: go
  affects: …provider-aws@v6.71.0
→ []
✓ list_global_security_advisories
  affects: …provider-archive@v2.8.1
→ []
✓ open · s3 · staging/…
✓ up
status = awaiting_approval
plan = { 0 add, 3 change, 0 destroy }
```

TURF PLAN · PULL REQUEST #7 — A COMMENT, THEN A REVIEW

Plan: 0 to add, 3 to change, 0 to destroy

```
~ aws_lambda_function.hello[0]
    source_code_hash: BgdFgZjt7YLjXpiX… → hfdhdiss6ibmMnYm…
~ aws_lambda_function.hello[1]
    source_code_hash: BgdFgZjt7YLjXpiX… → hfdhdiss6ibmMnYm…
~ terraform_data.release
    input.code: BgdFgZjt7YLjXpiX… → hfdhdiss6ibmMnYm…
```

The handler code changed. The function code hash is the only input that differs. Greeting, environment and URLs are unchanged. There are no output changes and no drift.

Approval

Verdict: approve. The `staging` policy approves any plan unless it destroys `aws_iam_role.lambda_exec` or deletes every function. This plan has no deletes, and the role and both functions remain.

`review · COMMENT · “The plan is ready to merge under the staging policy.”`

From run 41251741913, 2026-10-31, trimmed: the providers table, the invocations and the `turf` block are omitted.

PART II
THE LOOP

04

OPEN
PLAN
COMMENT
REVIEW
MERGE
APPLY
VERIFY
PROMOTE

THE RULE

Rules in plain language

The approval rule is a paragraph. The Environment names which one.

`policy/approval.md` holds one rule per Environment. Staging approves almost anything, with two exceptions it names. Production approves creates and in-place updates, and nothing else. The GitHub Environment sets `TURF_APPROVAL_POLICY`, the run context carries it, and the same runbook reads a different paragraph in each environment.

Underneath sit the deployment skill's own rules, which the agent loads first: never a destroy plan, never a warning it cannot explain, never a plan the providers rule refused. The verdict is written the same way every time, `approve` or `refuse` and the rule that decided it, so the record reads the same whether a person or the next run looks at it.

```
## `staging`

Approve any plan, including replacements and deletes, except one that would:

- destroy the shared execution role (aws_iam_role.lambda_exec), or
- delete every function in the fleet (a plan whose deletes leave no
  aws_lambda_function.hello instance).

## `production`

Approve only a plan made of creates and in-place updates:

- no replacement (replace must be zero),
- no delete of any kind, including a shrink of the fleet,
- the first rollout's creates are allowed, since nothing exists yet.

Any other plan is refused: report it with the rule that refused it, say that a person
has to run the apply for this environment by hand, cancel the convergence, and apply
nothing.
```

policy/approval.md, the two rules, lightly trimmed.

staging GITHUB ENVIRONMENT

TURF_APPROVAL_POLICY	staging
AWS_ROLE_ARN	…:role/…-staging
TF_STATE_KEY	staging/lambda-fleet.tfstate
TF_VAR_fleet_size	2
approves	almost anything; two exceptions

production GITHUB ENVIRONMENT

TURF_APPROVAL_POLICY	production
AWS_ROLE_ARN	…:role/…-production
TF_STATE_KEY	production/lambda-fleet.tfstate
TF_VAR_fleet_size	3
approves	creates and in-place updates

Figure 3. The Environment selects the rule. Its variables, its role, its state key, and the paragraph the run obeys.

PART II
THE LOOP

05

- OPEN
- PLAN
- COMMENT
- REVIEW
- **MERGE**
- **APPLY**
- VERIFY
- PROMOTE

DAY 2 · THE MERGE

The apply under the rule

A merge to `main` starts the staging apply. It plans again, as every apply must, and then does what the classic pipeline cannot: it compares.

The runbook finds the plan comment the pull request carries and compares the new plan's changes and attributes with the reviewed plan's `turf` block. The same changes, the same values: the plan matches the reviewed plan, and the rule approves it. Then `apply` with the plan's digest. The digest is the approval, of that plan and no other: a plan that changed between the review and the apply carries a different one, and the runbook refuses it with the difference in the report.

The convergence runs in the engine, in rounds: one here, since nothing is deferred. The agent follows it: `awaiting_actions` when the configuration asks something of it, `converged` when nothing is left, then the report: the counts, each round, the action's progress lines, the outputs, and the label that lets production start.

```
Apply. If the verdict is refuse: post the report with "kind": "refused", then
cancel({ cause: "<the rule that refused>" }), and stop; nothing is applied. If the verdict
is approve: apply({ convergence_id, plan_digest }) with the digest up returned, then converge:

- awaiting_approval (a later round): review it exactly as in step 5 under the same policy;
  approve it by its own digest, or refuse as above.
- awaiting_actions: perform every open request with your tools, as its prompt says, and
  answer all of them in one apply({ convergence_id, action_results }). … Never report a
  success you did not observe.
- converged: read output({}), post the report with "kind": "apply", and add the label
  the workflow names.
- failed, stalled, exhausted, stale, cancelled, error: post the report with
  "kind": "error" and the detail from rounds[], and cancel if the convergence is still paused.
```

`.github/workflows/shared/turf-runbook.md`, step 7, trimmed.

```
TOOL CALLS · TURF
✓ up
outcome = awaiting_approval · round 1
plan.digest = sha256:f1d9d24f…
plan = { add 0, change 3, destroy 0 }
plan.invocations[0] = { action.turf_action.verify,
  terraform_data.release, after_update, halt }
✓ apply · { convergence_id: "p-cb1df7ac",
            plan_digest: "sha256:f1d9d24f…" }
outcome = awaiting_actions
… the verify action, chapter 6 …
✓ apply · { convergence_id, action_results }
outcome = converged
✓ output · urls, greeting, function_names
```

TURF APPLY (STAGING) · PR #7 THE APPLY COMMENT

Approval

Verdict: approve. Rule: the `staging` policy approves any plan unless it destroys `aws_iam_role.lambda_exec` or deletes every function. This plan has no deletes. Matches the reviewed plan: the changes and updates are the same as in the earlier plan comment.

Apply

Round 1: applied 3, deferred 0
Outputs: `function_names` turf-hello-staging-0, turf-hello-staging-1 · `greeting` Hello from staging! · `urls`: the two function URLs above

`label · turf/applied-staging`

From run 41254055719, 2026-10-31: the Approval and Apply sections, the action's progress lines left to chapter 6.

PART II
THE LOOP

06

- OPEN
- PLAN
- COMMENT
- REVIEW
- MERGE
- **APPLY**
- **VERIFY**
- PROMOTE

INSIDE THE APPLY

The agent inside the apply

The verification is a turf action: a step the configuration asks of the agent, carried out with the agent's own tools, inside the apply.

An overlay file lays the action over the configuration at every plan: after the release is created or updated, `verify` fires. The engine stops at `awaiting_actions` with the prompt and the inputs. The agent fetches each URL with curl, checks the body, and answers with `apply` and the action's result. `on_failure = halt`: a failed check halts the convergence, the report says why, and the label production waits for is never set. Honest by construction: the first apply of this fleet answered `succeeded: false` when the sandbox refused the agent's curl loop.

```
action "turf_action" "verify" {
  config {
    prompt = <<-EOT
      The fleet was just deployed or patched. Fetch every URL in inputs.urls
      with curl. Each must answer HTTP 200 with a JSON body whose "message"
      equals inputs.greeting and whose "environment" equals inputs.environment. …
    EOT
    inputs = { greeting = caller.output.greeting, environment = caller.output.environment,
               urls = caller.output.urls }
  }
}
action_trigger "verify" {
  target     = terraform_data.release
  events     = [after_create, after_update]
  actions    = [action.turf_action.verify]
  on_failure = halt
}
```

infra/rollout.turf.hcl, the prompt trimmed. Turf lays it over main.tf; tofu and terraform ignore it by its name.

```
TOOL RESULT · apply(plan_digest)
outcome = awaiting_actions
action_requests[0] = {
  key = "action/1/action.turf_action.verify@after_update@…"
  if_failed = "A failure halts terraform_data.release …"
  inputs = { environment = "staging",
             greeting = "Hello from staging!",
             urls = [ "https://533apcrq…", "https://op3tstef…" ]
}
BASH · ONE PLAIN CURL PER URL
$ curl -sS -m 15 -w '\n%{http_code}' https://533apcrq….on.aws/
{"message":"Hello from staging!","function":"…-staging-0",
 "environment":"staging","region":"us-west-2"}
200
TOOL CALL · apply(action_results)
action_results = { "action/1/…@…": { succeeded: true,
  progress: [ "GET …533apcrq… → 200, message "Hello from
    staging!", environment "staging" (turf-hello-staging-0)",
    "GET …op3tstef… → 200, … (turf-hello-staging-1)" ] } }
outcome = converged · rounds[0] = { applied 3, invoked 1 }
```

THE AGENT — IN THE REPORT

Apply

Round 1: applied 3, deferred 0
Turf action `action.turf_action.verify`:

- GET …533apcrq… → 200, message "Hello from staging!", environment "staging" (turf-hello-staging-0)
- GET …op3tstef… → 200, message "Hello from staging!", environment "staging" (turf-hello-staging-1)

`label · turf/applied-staging`

From run 41254055719, 2026-10-31, the agent job's log, trimmed; the URLs are shortened.

PART II
THE LOOP

07

OPEN
PLAN
COMMENT
REVIEW
MERGE
APPLY
VERIFY
PROMOTE

DAY 2 · PRODUCTION

Promotion by Environment

Production runs the same configuration, from the same commit, under a stricter rule. Nothing is copied. The Environment supplies the rest.

The production workflow starts on `workflow_run`, when the staging workflow completes successfully on `main`. A gh-aw run ends green whenever its agent finishes, so success alone is not enough: a label check in a pre-agent step requires `turf/applied-staging` on the merged pull request, which the staging agent adds only when its apply converged. Without it, production waits, and the log says so.

From there the run is the staging run again, with the production Environment's values: its role, its state key, a fleet of three, and the rule that allows creates and in-place updates only. The plan comment on the pull request is a staging plan, and the agent says so: “the only earlier plan comment is for `staging`, with 2 functions and 3 changes … It is not the same plan”. The production rule decided on its own terms: 4 updates, 0 replacements, 0 deletes.

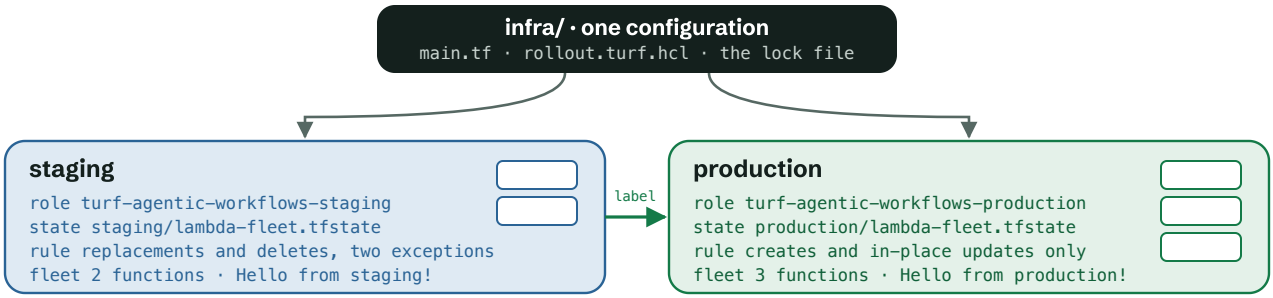


Figure 4. One configuration, two Environments. Each Environment brings its own role, state key, fleet and rule; the staging label is what lets production begin.

```yaml
on:
  workflow_run:
    workflows: ["Turf apply (staging)"]
    conclusion: [success]
environment: production
pre-agent-steps:
  - name: Require a converged staging apply of this commit
    run: |
      PR=$(gh pr list --state merged --search "$HEAD_SHA" --json number --jq '.[0].number // empty')
      if [ -n "$PR" ] && gh pr view "$PR" --json labels --jq '.labels[].name' \
           | grep -qx 'turf/applied-staging'; then
        echo "Staging converged for #$PR."
      else
        echo "::error::Staging has not converged for $HEAD_SHA; production waits."
        exit 1
      fi
```

`.github/workflows/turf-apply-production.md`, trimmed to the trigger and the label check.

PART II
THE LOOP

08

- OPEN
- **PLAN**
- COMMENT
- **REVIEW**
- MERGE
- APPLY
- VERIFY
- PROMOTE

WHAT REFUSAL LOOKS LIKE

Refused

A refusal is a review, not a red X. Two pull requests show the two rules refusing.

Pull request #5 renames the shared execution role. A name change forces a replacement, and the staging rule names that role as the one thing it will not destroy. The plan comment shows the replacement, the verdict quotes the rule, the review requests changes, and the convergence is cancelled with the lock released. Nothing is applied.

Pull request #6 adds the `hashicorp/vault` provider pinned at 5.4.0. The providers rule found GHSA-gmm6-j2g5-r52m in GitHub's Advisory Database, high severity, fixed in 5.5.0, and refused before `open`: nothing planned, no lock taken, and the review says what to bump.

TURF PLAN · #5 · THE DIFF BLOCK, TRIMMED

```
Plan: 2 to add, 2 to change, 2 to destroy (2 replaced)
∓ aws_iam_role.lambda_exec (must be replaced: name)
    name: …-staging-exec → …-staging-lambda-exec
    arn: …:role/…-staging-exec → (known after apply)
∓ aws_iam_role_policy_attachment.lambda_policy
    role: …-staging-exec → …-staging-lambda-exec
~ aws_lambda_function.hello[0]
    role: …:role/…-staging-exec → (known after apply)
~ aws_lambda_function.hello[1]
    role: …:role/…-staging-exec → (known after apply)

Verdict: refuse
```

TURF PLAN · PR #5 — THE REVIEW

Refused under the staging approval rule (`policy/approval.md`): the plan would destroy the shared execution role `aws_iam_role.lambda_exec`. Its name changes from `turf-hello-staging-exec` to `turf-hello-staging-lambda-exec`, which forces a replacement. The policy attachment is replaced and both functions are updated to the new role. The plan is 2 to add, 2 to change, 2 to destroy. The providers rule passed (no advisories). A person needs to review and apply this change by hand, or keep the existing role name.

REQUEST_CHANGES

From run 41238948486, 2026-10-31. The review is quoted whole.

TURF PLAN · #6 · THE PROVIDERS TABLE

Provider	Version	Advisories	Verdict
hashicorp/archive	2.8.1	none	allow
hashicorp/aws	6.71.0	none	allow
hashicorp/vault	5.4.0	1 (high)	deny
GHSA-gmm6-j2g5-r52m / CVE-2025-13357, fixed in 5.5.0			

```
Refused by providers rule 3 (Advisories).
The workspace was not opened, no plan was made,
and no state lock was taken.
```

TURF PLAN · PR #6 — THE REVIEW

Refused by `policy/providers.md` rule 3 (Advisories). `hashicorp/vault` 5.4.0 has a high-severity advisory (GHSA-gmm6-j2g5-r52m / CVE-2025-13357) that is fixed only in 5.5.0, above the pinned version. Nothing was opened or planned. Bump the vault provider to 5.5.0 or later and update `.terraform.lock.hcl`.

REQUEST_CHANGES

From run 41238989000, 2026-10-31. The review is quoted whole; the table is the comment's, reset in monospace.

09

OPEN
PLAN
COMMENT
REVIEW
MERGE
APPLY
VERIFY
PROMOTE

THE GUARANTEES

Whole convergence, in one run

One run is one convergence, held in the engine from `up` to `converged`, with every round's plan approved before it runs.

A convergence is rounds. The engine plans what it can know, the agent approves the round by its digest, the engine applies it, and if a value is known only after that apply, the next round plans what needed it. The fleet here converges in one round; a configuration with a dependency on an apply-time value takes two, in the same run, under the same rule. The classic pipeline has no place for a second round.

The engine refreshes before it plans and reports drift beside the plan; the agent classifies it: the provider catching up on a value it now reads differently, or a change made outside the configuration, named in the report. The state lock is taken by `open` and released when the server stops. When a run ends badly, it says so: a verify that answers `succeeded: false` halts the convergence, the label is withheld, production waits, and the report names the reason. The pull request's `turf` blocks are the memory between runs.

The configuration never names an environment. The Environment supplies the variables, the role, the state key and the rule, so one body of code serves staging and production, and the record says which it was.

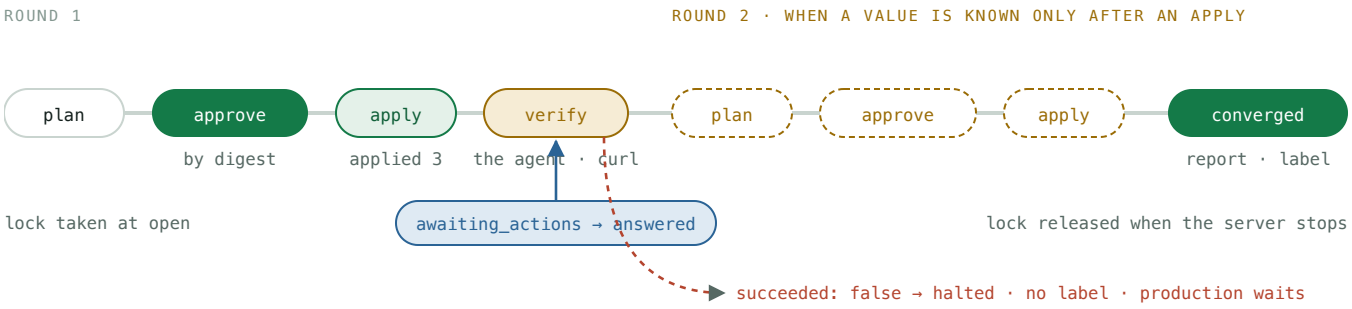


Figure 5. A run. Rounds until nothing is deferred, each approved by its own digest; the verify action is the agent's part; a failure halts, and leaves a record instead of a label.

PART III
THE GUARANTEES

10

- OPEN
- PLAN
- COMMENT
- REVIEW
- MERGE
- APPLY
- VERIFY
- PROMOTE

THE GUARANTEES

Safety, in layers

Two systems, each with its own layers, and the agent between them with the least it needs.

GitHub's layers hold the agent: what it may reach, what it may call, what it may write, and with which token. Turf's layers hold the change: which plan, under which rule, with which credentials, and what the configuration may ask. Nothing in the list is a feature flag. Each is a line in a file in the repository, and the paper has shown most of them.

GitHub's layers

AROUND THE AGENT

- ✓ A read-only token. The agent cannot write to the repository, the pull request or the issue tracker.
  `permissions: contents: read · pull-requests: read`
- ✓ A sandbox behind the Agent Workflow Firewall: only the domains the frontmatter allows.
  `network.allowed: defaults, github, host.docker.internal, the function URLs`
- ✓ Safe outputs: the comment, the review and the label are posted by a separate job, after threat detection.
  `safe-outputs: add-comment · submit-pull-request-review · add-labels`
- ✓ The MCP gateway enforces the tool list: a plan run cannot call `apply`.
  `mcp-servers.turf.allowed`
- ✓ GitHub Environments hold the variables, the role and the rule's name, per environment.
  `environment: staging · environment: production`
- ✓ The workflow and its compiled lock file are both committed, and reviewed like any code.
  `gh aw compile · .lock.yml`

Turf's layers

AROUND THE CHANGE

- ✓ Approval by digest: `apply` names the plan it approves, and no other plan runs.
  `apply({ convergence_id, plan_digest })`
- ✓ A rule per environment, in plain language, and the verdict written with the rule that decided.
  `policy/approval.md · TURF_APPROVAL_POLICY`
- ✓ The providers rule: source, pin and advisories, checked before anything is opened.
  `policy/providers.md`
- ✓ Cloud credentials minted by OIDC for one hour, in the engine's process, never in the agent's.
  `assume-role-with-web-identity · the pre-agent step`
- ✓ The deployment skill's rules underneath: never a destroy plan, never a warning you cannot explain.
  `skill_deployment`
- ✓ One convergence per run, cancelled on refusal, the lock released on the way out; `destroy`, `taint`, `import` and `invoke` are not the workflow's to run.
  `cancel({ cause }) · the post-step`

Figure 6. Safety, in layers. Each line names the file or the call where the layer lives.

Run it

The pipeline in this paper is public: **github.com/turfbuild/turf-agentic-workflows**. Fork it, and:

1. Apply `setup/` with Turf, from your laptop: the two GitHub Environments with their variables, one AWS role per environment trusting the repository's OIDC subject, and the state bucket, as Terraform.
2. Set `ANTHROPIC_API_KEY` as a repository secret, or switch the engine to Copilot or Codex.
3. `gh aw compile`, commit the workflows with their lock files, and open a pull request that touches `infra/`.

Or import the component into a workflow of your own, since gh aw imports resolve across repositories: `imports: [turfbuild/turf-agentic-workflows/.github/workflows/shared/turf.md@main]`.

GET TURF

Download it. Run it.

Turf is available now, as a download. No form, no login. Free for noncommercial use.

```
$ curl -fsSL https://get.turf.build/install.sh | bash
```

Commercial use, including a company's internal use, requires a license. Companies evaluate Turf under a free pilot: turf.build/licensing. Turf sends anonymous usage telemetry to Turf LLC by default: the version, the platform, counts of resources, changes and runs, and an anonymous installation id. It never sends resource contents, credentials, state, variable values or names. Turn it off with `turf telemetry off`, or with `TURF_TELEMETRY=off` or `DO_NOT_TRACK=1` in the engine's environment; the full field list and the endpoint are at turf.build/telemetry.

About Turf

Turf™ is a self-hosted, autonomous infrastructure-as-code tool, built on OpenTofu™. Every change, from a person or an agent, is a change plan that is checked, approved and recorded. Turf runs on a laptop, in a cluster, or on a CI runner, and any agent drives it through its MCP server.

turf.build

Turf™ is a trademark of Turf LLC. Terraform® is a registered trademark, and HashiCorp Vault a trademark, of HashiCorp, Inc. OpenTofu™ is a trademark of the Linux Foundation. GitHub®, GitHub Actions, GitHub Copilot and GitHub Agentic Workflows are trademarks of GitHub, Inc. AWS and AWS Lambda are trademarks of Amazon.com, Inc. or its affiliates. Claude is a trademark of Anthropic, PBC. Turf is not affiliated with, endorsed by, or sponsored by HashiCorp, IBM, the Linux Foundation, the OpenTofu project, GitHub, Amazon or Anthropic; every mark is used only to describe interoperability. Quotations from HashiCorp's tutorial “Automate Terraform with GitHub Actions”, from OpenTofu's documentation and from GitHub's documentation are attributed in the text. All other marks belong to their owners.

Pull Request to Production, v0.1, 2026-10-31.